

# ใบงานที่ 6 การสร้างปัญญาประดิษฐ์เล่นเกม Tic-Tac-Toe

## วัตถุประสงค์การเรียนรู้

1. เขียนคำสั่งภาษา Python เพื่อจำลองสภาพแวดล้อม (Environment) ของเกม Tic-Tac-Toe ได้
2. เขียนคำสั่งสร้างตัวแทน (Agent) และประยุกต์ใช้อัลกอริทึม Q-Learning ได้
3. เขียนคำสั่งเพื่อคำนวณและอัปเดตค่าความรู้ (Q-Table) ตามสมการ Bellman Equation ได้
4. อธิบายการทำงานของ Epsilon-Greedy Strategy ในการตัดสินใจของ AI ได้
5. พิจารณาเปรียบเทียบประสิทธิภาพระหว่างการฝึกฝนแบบ Single Agent และ Dual Agent ได้

## เครื่องมือและอุปกรณ์

1. เครื่องคอมพิวเตอร์ที่ติดตั้งโปรแกรม PyCharm IDE
2. เอกสารใบความรู้หน่วยที่ 6
3. แบบฟอร์มบันทึกผลการปฏิบัติงาน (ท้ายใบงาน)

# ลำดับขั้นตอนการปฏิบัติงาน

คำชี้แจง: ให้ผู้เรียนสร้างไฟล์ main.py และเขียนโค้ดตามขั้นตอนต่อไปนี้

## ขั้นตอนที่ 1: เตรียมสภาพแวดล้อมและเกม (Environment Setup)

1. Import Library: นำเข้า numpy, random, และ defaultdict จาก collections เพื่อใช้จัดการข้อมูลและตาราง
2. สร้าง Class TicTacToe:
  - สร้างเมธอด `__init__`: กำหนดกระดานขนาด 3x3 ด้วย np.zeros และกำหนดผู้เล่นเริ่มก่อนเป็น 1
  - สร้างเมธอด `reset`: สำหรับล้างกระดานเมื่อจบเกม
  - สร้างเมธอด `available_actions`: คืนค่า List ของช่องที่ว่างอยู่ (ที่เป็นเลข 0)
  - สร้างเมธอด `make_move`: รับค่า action (0-8) แปลงเป็นพิกัดแถว/คอลัมน์ด้วย divmod และลงหมากในกระดาน
  - สร้างเมธอด `check_winner`: ตรวจสอบผู้ชนะ 8 ทิศทาง (แนวนอน, แนวตั้ง, แนวทแยง)
  - สร้างเมธอด `print_board` แสดงผลกระดานเกม

## ขั้นตอนที่ 2: สร้างสมองของ AI (The Q-Learning Agent)

1. สร้าง Class QLearningAI
2. กำหนดค่า Hyperparameters (`__init__`):
  - alpha (Learning Rate) = 0.1
  - gamma (Discount Factor) = 0.9
  - epsilon (Exploration Rate) = 0.2
  - สร้าง q\_table ด้วย defaultdict
3. สร้างฟังก์ชัน `choose_action` (Epsilon-Greedy)
  - สุ่มตัวเลข: หากน้อยกว่า epsilon ให้เลือกเดินแบบสุ่ม (Exploration)
  - หากมากกว่า: ให้เลือกเดินช่องที่มีค่า Q สูงที่สุดจาก q\_table (Exploitation)
4. สร้างฟังก์ชัน `update` (Bellman Equation):
  - เขียนสมการ:  $OldQ + Alpha * (Reward + Gamma * MaxNextQ - OldQ)$  เพื่อปรับปรุงค่าในตาราง

### ขั้นตอนที่ 3: การฝึกฝนโมเดล (Training Loop)

1. สร้างฟังก์ชัน train: กำหนดค่าเริ่มต้น episodes=1000
2. เขียน Loop การฝึก:
  - วนลูปตามจำนวน episode
  - ในแต่ละรอบ ให้ AI เล่นแข่งกับตัวเอง (Self-Play) จนจบเกม
  - การให้รางวัล (Reward): ชนะได้ +1, แพ้ได้ -1, เสมอหรือเดินทั่วไปได้ 0
  - เรียกใช้ ai.update() เพื่อสอน AI ทุกครั้งที่มีการเดิน หรือจบเกม
3. Return AI: ส่งค่า AI ที่ฝึกเสร็จแล้วกลับออกมา

### ขั้นตอนที่ 4: ทดสอบเล่นกับมนุษย์ (Deployment)

1. สร้างฟังก์ชัน play\_against\_ai:
  - รับค่า AI เข้ามา
  - วนลูปรับค่า Input จากผู้เล่น (ตำแหน่ง 1-9)
  - ให้ AI (X) ตัดสินใจเดินโดยกำหนด training=False (เอาจริง ไม่เดินมั่ว)
  - แสดงผลกระดานและประกาศผู้ชนะเมื่อจบเกม
2. เขียนส่วน main: เรียกใช้ train() และตามด้วย play\_against\_ai()

## แบบบันทึกผลการปฏิบัติงาน

### ส่วนที่ 1: ความเข้าใจในตัวแปรและสมการ (Code Understanding)

คำชี้แจง: จงอธิบายความหมายของตัวแปรในสมการ update ที่ท่านได้เขียนลงไป

1. alpha (Learning Rate) ในโค้ดนี้กำหนดค่าไว้ที่ 0.1 หมายความว่าอย่างไร ?

.....

.....

2. gamma (Discount Factor) ถ้าเราปรับค่านี้เป็น 0 จะส่งผลกระทบต่อพฤติกรรมของ AI ในการวางแผนล่วงหน้า อย่างไร ?

.....

.....

3. epsilon (Exploration Rate) ในฟังก์ชัน choose\_action มีไว้เพื่อวัตถุประสงค์อะไร ?

.....

.....

### ส่วนที่ 2: การเปรียบเทียบวิธีการฝึกฝน (Algorithm Comparison)

คำชี้แจง: ให้ผู้เรียนทดลองรัน Code ด้วยฟังก์ชัน train (แบบเก่า) และ train\_new (แบบใหม่) โดยกำหนดจำนวนรอบฝึกที่ 10,000 รอบเท่ากัน แล้วบันทึกผลการแข่งขันกับ AI จำนวน 5 ตา

```
# ให้ผู้เรียนเพิ่มฟังก์ชันนี้เข้าไปใน code
```

```
def train_new(episodes=10000):
```

```
    ai_x = QLearningAI()
```

```
    ai_o = QLearningAI()
```

```
    game = TicTacToe()
```

```
    for ep in range(episodes):
```

```
        game.reset()
```

```
        st_x, act_x = None, None
```

```
        st_o, act_o = None, None
```

```
while True:
```

```
    if game.current_player == 1: # ตา X
        state = ai_x.state_key(game.board)
        action = ai_x.choose_action(game.board, training=True)
        if st_x: ai_x.update(st_x, act_x, 0, state)
```

```

        game.make_move(action)
        st_x, act_x = state, action
```

```

        winner = game.check_winner()
```

```
        if winner is not None:
```

```
            r_x = 1 if winner == 1 else -1 if winner == -1 else 0
            r_o = 1 if winner == -1 else -1 if winner == 1 else 0
            ai_x.update(st_x, act_x, r_x, None)
            if st_o: ai_o.update(st_o, act_o, r_o, None)
            break
```

```
    else: # ตา O
```

```
        state = ai_o.state_key(game.board)
        action = ai_o.choose_action(game.board, training=True)
        if st_o: ai_o.update(st_o, act_o, 0, state)
```

```

        game.make_move(action)
        st_o, act_o = state, action
```

```

        winner = game.check_winner()
```

```
        if winner is not None:
```

```
            r_x = 1 if winner == 1 else -1 if winner == -1 else 0
            r_o = 1 if winner == -1 else -1 if winner == 1 else 0
            if st_x: ai_x.update(st_x, act_x, r_x, None)
            ai_o.update(st_o, act_o, r_o, None)
            break
```

```
return ai_x
```

1. การทดลองที่ 1: ฟังก์ชัน train แบบเก่า (เล่นคนเดียว)

- สมมติฐาน: AI ใช้สมองกลคนเดียวเล่นเป็นทั้ง X และ O
- กำหนด episodes = 10,000
- ผลการแข่งขัน 5 ตา (เรา vs AI):  
ชนะ ..... ครั้ง / แพ้ ..... ครั้ง / เสมอ ..... ครั้ง
- สังเกตพฤติกรรม AI:

.....

2. การทดลองที่ 2: ฟังก์ชัน train\_new แบบใหม่ (แยกสมอง 2 ตัว)

- สมมติฐาน: แยก AI เป็น 2 ตัว (X และ O) แข่งกันเองเพื่อเอาชนะ
- กำหนด episodes = 10,000
- ผลการแข่งขัน 5 ตา (เรา vs AI):  
ชนะ ..... ครั้ง / แพ้ ..... ครั้ง / เสมอ ..... ครั้ง
- สังเกตพฤติกรรม AI:

.....

ส่วนที่ 3: วิเคราะห์ผล (Critical Thinking)

1. ทำไมเมื่อแยก AI เป็น 2 ตัว (X และ O) แข่งกันเองเพื่อเอาชนะถึงเล่นเก่งกว่า AI ใช้สมองกลคนเดียวเล่นเป็นทั้ง X และ O ทั้งที่ฝึกจำนวนรอบเท่ากัน ?

.....

.....

.....